

ROBO 5000 – Fall 2026

Intro to Robotics

Challenge Problem 1 Report

Alex Smith & Ben Falco

Introduction to the Problem:

For Challenge Problem 1 we were tasked with creating a program that could create and control the movements of multiple instances of 3 different robot types (Drone, Humanoid, Differential Drive) from their starting locations to their goal locations without any collisions. We also had to produce a visual representation of the grid and the robots on it for each timestep and an interface allowing a user to go back and forth between time steps.

1 Approach to the Problem:

1.1 Separation of Responsibilities

When we first met to discuss the assignment we broke it down into 4 main parts: setting up the framework, encoding the robots with the greedy movement, the visualization/user interaction, and the report.

After discussing the our ideas behind the general outline of our logic of the 4 individual parts, Ben put together a starting setup for the framework so we had a jumping off point. Then, on their own Ben worked on the greedy movement, and Alex worked on the Visualization.

1.2 Framework:

Our code framework consists of five classes total. To represent the robots we created a Robot base class which contains functionality which is common across all robot classes such as storing positions and goals and getting desired movements. We then created a subclass of Robot for each robot type, including Drone, Humanoid, and Differential Drive. The only functionality these subclasses implement is the `can_coexist` function which describes whether one robot can occupy the same cell as another robot.

The fifth class is the Grid class. This class manages initializing robots, visualization, and performing greedy movement. The `run.py` script instantiates a Grid object and handles user input to run the code.

1.3 Greedy Movement:

The greedy movement is executed as follows

1. All robots which are already at their target are ignored
2. All robots which can reach their target in one step are moved to their target
3. A dictionary mapping hot cells (cells where at least one robot desires to move to) to robot id's and the robots target distance is created
4. The hot cell dictionary is iterated over so that the best candidate for each cell is moved. The best movement candidate is the robot with the largest distance to its target which can move without causing a collision
5. Robot's which desire to swap locations are moved if this can be done without causing collisions
6. Steps 4 and 5 are repeated until no more movements occur (this usually takes two cycles)

7. If no movements occurred in steps 2-6, move all robots with at least two adjacent robots. This step is explained in the Loop Cases section below
8. Any robots which were not moved in steps 2-7 are commanded to wait at their current location

1.4 Visualization:

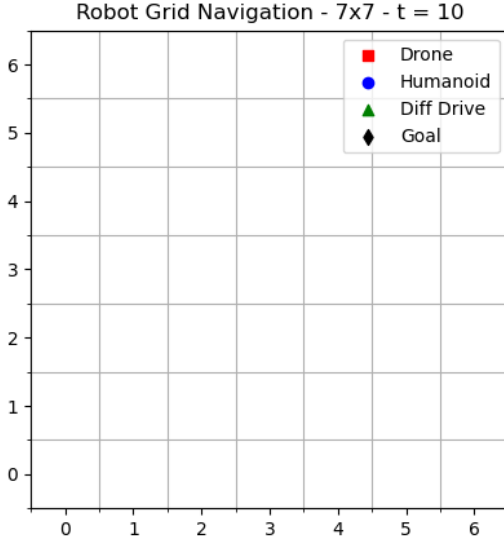


Figure 1: An empty 7x7 grid

We had to use various shapes and colors in order denote different types of robots in our grid. So when it came time to plot the robots on the grid our code checks the type of robot and then assigns it a color and shape corresponding to the type. After scatter plotting each robot and it's goal, we plot a straight line from the robot to its goal.

We also added a feature to ensure that you could see each individual robot and goal easier when there were multiple scatter points in the same cell of the grid. We used the `np.random` library in order access two random numbers between $[-0.3, 0.3]$. These floats were then added to the goal position and robot position on the grid. The offset serves two purposes: spread out the items within a cell so they aren't covering each other and because the same floats were added both to the robot and the goal it gave another visual cue connecting the two and differentiating it from other pairs.

The visualization took on different forms throughout the course of its creation. But it started off with just the base of an $n \times n$ grid produced with the 'matplotlib.pyplot' library. We wanted the markers that denote the column and row value of each cell to be in the center of the cell and not on a grid line. This meant that the x and y limits had to be set from $[-0.5, (n - 0.5)]$ to end up with cells (0,0) all the way to (n-1, n-1).

Major (every step of value 1) and minor (every step of value 0.5) ticks had to be set in order to be able to set the grid on the minor ticks and frame each 1x1 cell around the major ticks

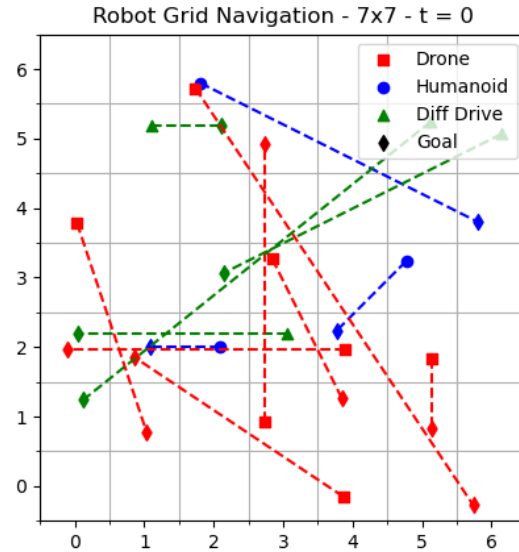


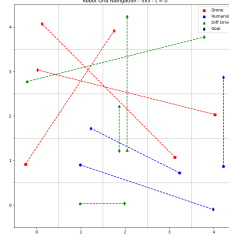
Figure 2: A grid full of robots, goals and paths

Finally, we used the 'mpl.connect' library by matplotlib to take in keyboard inputs while the window of the figure was still open to progress forwards and backwards through the time steps. This required the creation

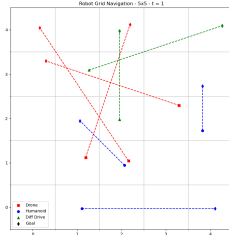
of an 'on_key' function that would interpret the keystroke and perform the step forward and step backward if the left or right arrow respectively is pressed.

2 Our Solution's Results:

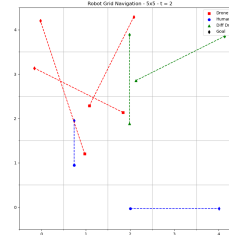
Our finished program can successfully work through all the steps from creation, to management, to visualization of the movements of a grid of various robots. Here is a collection of various time steps in a test run:



(a) $t=0$



(b) $t=1$



(c) $t=2$

Figure 3: $t=0 \dots t=2$

3 Challenges:

3.1 Interacting With the Command Line While a Plot is Open

At first we tried to receive the keyboard inputs to control the graph through the command line while the program was running. But this caused issues because the command line wouldn't take inputs while the plot was open because that part of the code wasn't resolved until the window was closed.

Our solution was to utilize the `mpl.connect` library as previously mentioned, with the use of `'ax.clear'` to remove the data on the grid opening up room for a new time step.

3.2 Producing an Accurate Legend

Initially our code was compiling all of the scatter plot data to be plotted all at once. But this was more complicated to pull off, so we moved to individually plotting each set of robot and goal. This introduced the issue of a legend with an item for each individual robot rather than each class of robot.

Our solution was to after the individual plotting, plot empty lists for each robot type and the goals. These could then be used as the entries for the legend.

4 Where we could expand:

4.1 Loop Cases

There is a very rare case where our method of movement could fail. This occurs when there is a loop of robots which all want to move in a loop together and there are multiple adjacent robots which want to move into the loop, shown in Figure 4. Currently when there is a loop we move all of the robots which have at least two adjacent robots. In order to solve this, we would need to do loop detection and only move the robots inside the loop. We decided performing loop detection was not worth the effort for a case which only occurs every few tens of thousands of random initial conditions.

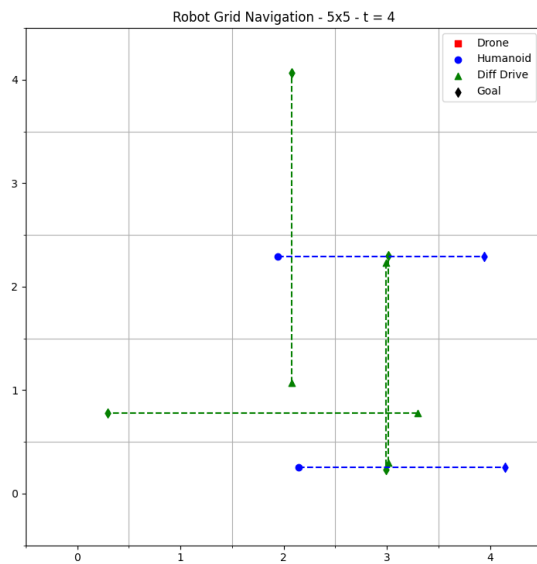


Figure 4: A case where there is a loop with an robot wishing to enter the loop

5 Conclusion:

We successfully created a program that can take in user input to create an $n \times n$ grid, instantiate $2 \times n$ instances of various subclasses of our Robot class with their respective goals, control their movement until they reach their goals while avoiding collisions, and visualize the entire process in a matplotlib window.

6 References:

trio of images : <https://tex.stackexchange.com/questions/222656/how-to-align-multiple-subfigures-in-a-grid>